

面向DXF与DWG工控图纸生成的 Text-to-CAD开源生态与架构演进研究报告

计算机辅助设计(CAD)领域的智能化变革正经历从传统的“离散几何图形生成”向“生成可编辑、具备工程约束的代码与矢量图纸”的范式转变¹。在工业制造、建筑施工及机械加工等实际场景中，DXF(Drawing Exchange Format)与DWG(Drawing File Format)作为二维矢量图纸交换的核心格式，承载着极其严谨的拓扑关系、图层分类、几何尺寸标注以及制造工艺约束⁴。相比三维网格或隐式场表达，二维工程图纸对数值精确度与边界闭合性有着近乎苛刻的要求，任何微小的几何坐标断裂或自相交都会直接导致激光切割、数控机床(CNC)加工或下料排版失败⁶。

目前基于自然语言指令(Text-to-CAD)自动生成高质量DXF/DWG图纸的技术路径，已从早期的端到端离散序列预测，演变为结合大语言模型(LLM)代码合成、工业级几何求解内核、自动化拓扑验证器与多模态视觉反馈的Agent智能系统¹。本报告围绕该领域成熟的开源库、计算几何引擎、AI大模型数据集及系统架构展开系统性解析，为构建工业级Text-to-CAD系统提供技术选型与落地参考。

二维矢量解析与图形渲染底层框架评估

生成式 Text-to-CAD 系统的高效运行依赖于底层的矢量图纸解析与操作框架。此类框架需具备对DXF/DWG文件结构的精准读写、实体图元提取、图层控制、拓扑转换及自动化渲染能力⁴。

ezdxf与Python矢量控制管线

在Python开源生态中，ezdxf 是操作DXF文件最成熟、功能覆盖最完整的核心库，支持从AutoCAD R12到最新的R2018+多版本DXF文件规范⁴。ezdxf 提供了对DXF文件内部段落结构的完整抽象，包括记录全局变量与单位设置的 HEADER 段、定义图层、线型与文字样式的 TABLES 段、存储可复用图块定义的 BLOCKS 段，以及包含直线(LINE)、圆弧(ARC)、圆(CIRCLE)和多段线(LWPOLYLINE)等几何对象的 ENTITIES 段⁴。

除了基础的图元读写功能外，ezdxf 具备高度扩展的附加组件生态⁴。通过 ezdxf.addons.drawing 模块，可将DXF中的矢量图元无缝渲染至Matplotlib或Qt后端，从而为视觉大模型(VLM)提供高精度的图像渲染支持⁴。在几何结构分析方面，ezdxf 可与 Python 计算几何库 Shapely 及 EdgeSmith 深度集成，将离散的线段与圆弧提取并组装为多边形(Polygon)拓扑结构，自动校验闭合轮廓与嵌套孔洞关系⁴。

DWG二进制转换与多语言底层库分析

由于 DWG 格式属于Autodesk的私有二进制协议，开源社区直接解析 DWG 的难度极高⁴。工业界主要采取两种技术路线：一种是采用GNU旗下的 C 语言库 LibreDWG 进行原生读写¹⁰；另一种则是利用 Open Design Alliance 提供的 ODA File Converter 工具，通过 ezdxf.addons.odafc 插件将 DWG 无损转换为 DXF 后再行处理⁴。

在多语言技术栈选型中，各开源解析库在协议授权、性能及平台支持上存在显著差异¹⁰。ezdxf 采用极其宽松的 MIT 协议，API 友好且极易与AI工作流结合，是构建大模型Agent系统的首选⁴。LibreDWG 虽然支持原生 DWG 读写，但受限于 GPLv3 协议，在商业闭源系统中集成存在法律合规风险¹⁰。在 C#/.NET 生态中，ACadSharp 提供了完全脱离 AutoCAD 软件依赖的强类型 API，支

持全版本 DXF 与 DWG 的读取¹⁰。而 C++ 领域的 dxflib 与 libdxfw 则凭借卓越的执行性能，广泛应用于 LibreCAD 等桌面端软件的核心渲染管线中¹⁰。

开源库名称	核心开发语言	DXF格式支持	DWG格式支持	开源授权协议	核心技术优势与适用场景
ezdxf	Python	全版本 (ASCII/Binary)	需挂载 ODAFC 转换	MIT	API抽象度极高，生态丰富，支持 Matplotlib 渲染与 Shapely拓扑分析 ⁴ 。
LibreDWG	C	全版本支持	原生读写支持	GPLv3	GNU官方底层解析库，解析性能极高，但存在 GPL协议传染性 ¹⁰ 。
ACadSharp	C# / .NET	全版本 (ASCII/Binary)	原生读写支持	MIT	强类型设计，结构映射完整，适合构建 Windows服务端高并发解析组件 ¹⁰ 。
libdxfw / dxflib	C++	全版本支持	原生读写支持	GPLv2 / GPLv3	LibreCAD核心内核，执行效率极高，适合端侧高性能矢量绘制与转换 ¹⁰ 。

参数化几何求解与代码化建模引擎

让大语言模型直接预测底层的绝对坐标(如输出成百上千个离散的 LINE 节点)在工程实践中被证明存在严重缺陷，例如极易产生坐标浮点截断误差、图元端点无法闭合以及无法实现复杂的圆角

与倒角操作⁷。现代化 Text-to-CAD 系统普遍采用“Code-as-CAD”(代码化建模)中间层机制，即引导大模型生成高层参数化建模脚本(如 build123d 或 CadQuery)，再由工业级几何求解内核编译生成精确的二维矢量图纸或三维模型¹。

build123d与CadQuery的核心机制对比

build123d 是基于 OpenCASCADE(OCCT)工业级几何内核开发的新一代 Python 参数化建模框架，被视为 CadQuery 的现代继承者¹⁵。build123d 重新设计了 Python 几何表达范式，提供了代数模式(Algebra Mode)与构建器模式(Builder Mode)两种编程模型¹⁵。代数模式采用无状态设计，通过重载运算符(如用 + 表示布尔并集，- 表示布尔差集，* 表示空间位置变换)来实现高度紧凑的表达¹⁵；构建器模式则利用 Python 的上下文管理器(with BuildPart(), with BuildSketch())来模拟人类设计师的逐步建模历史¹⁵。

在二维矢量草图构建方面，build123d 提供了 Line、JernArc、PolarLine、RectangleRounded 及 make_hull 等丰富的1D与2D几何图元类，并支持极其强大的选择器(Selectors)功能¹⁵。开发者或 AI大模型可以通过列表过滤与几何属性选择器(例如按长度、方向或位置选择 Edge)，精准地施加 fillet(圆角)或 chamfer(倒角)操作，并将最终生成的 2D Sketch 直接导出为工业级 DXF 文件¹⁵。相比之下，CadQuery 采用了基于方法链(Method Chaining)的流式接口¹⁶。虽然方法链在简短脚本中表现优异，但在复杂几何逻辑中，较长的方法链容易增加 LLM 生成代码时的语法断裂概率¹⁶。此外，build123d 对类型提示(Type Hints)和 VS Code 插件(如 ocp-vscode)提供了原生支持，能够实现毫秒级的实时几何预览与调试，这显著提升了 Agent 系统的闭环验证效率¹⁶。

维度特性	代码化建模范式 (build123d / CadQuery)	离散坐标直接生成范式 (Native DXF Tokens)
几何拓扑保障	由工业级OCCT内核保证边界闭合与连续性 ¹⁵	依赖模型绝对坐标预测，极易出现离散微小缝隙 ⁷
高级拓扑操作	原生支持选择器进行 Chamfer、Fillet及Boolean运算 ¹⁷	无法进行复杂相切圆弧计算与边缘倒角 ⁷
参数化与可编辑性	高度参数化，修改变量即可重新生成整张图纸 ¹⁵	缺乏参数逻辑，仅为静态坐标集合 ⁷
模型生成难度	需模型掌握代码语法，但代码上下文容错率高 ¹	生成格式简单，但数值精确度要求极其苛刻 ⁷
输出文件质量	自动优化图元，符合严格的工业制作标准 ¹⁵	容易包含冗余节点与自相交无效拓扑 ⁷

Text-to-CAD模型范式与开源数据生态

Text-to-CAD 的学术研究与工程实践在过去几年中经历了显著的范式演进，主要形成了离散

Token 序列生成、高层代码合成以及基于 Agent 的验证闭环三大技术流派¹。

从离散 Token 预测到可执行代码合成

早期的代表性工作如 **Text2CAD** (NeurIPS 2024 Spotlight) 将 3D/2D CAD 模型构建过程抽象为类似文本 Token 的操作序列 (包括草图绘制与挤出操作等)⁷。为了训练自回归 Transformer 网络, 该团队构建了首个包含 ~170K CAD 模型和 ~660K 自然语言标注的数据集¹⁹。该数据集利用视觉语言模型 (LLaVA-NeXT) 生成形状描述, 并由大语言模型 (Mixtral-50B) 扩展出从抽象概念 (L0) 到专家级几何细节 (L3) 的多层级 Prompt¹⁹。然而, 由于该范式将连续几何坐标量化为离散 Class Label (如 8-bit 量化), 导致生成的矢量边界常发生微小抖动与开裂, 且由于缺乏几何指针机制 (Pointer Mechanism), 无法支持对特定边施加倒角等复杂操作⁷。

为了解决离散序列生成的瓶颈, 后续研究迅速转向高层代码合成¹。**Text-to-CadQuery** 研究将 Text2CAD 数据集的 JSON 结构重新标注并扩充为 170,000 个 CadQuery Python 代码对¹。该方案利用 Gemini 2.0 Flash 将几何逻辑转化为代码, 并在后台构建了隔离的 Python 执行沙盒⁸。当代码运行报错时, 沙盒提取 Traceback 错误信息并反馈给模型进行自我纠错 (Self-Correction), 将代码一次性执行成功率从 53% 大幅提升至 85%⁸。微调后的 Qwen2.5-Coder-32B 模型在 Exact Match 指标上达到了 69.3%, 几何 Chamfer Distance 降低了 48.6%¹。在此基础上, **CAD-Coder** 进一步引入了多步逻辑推理机制与几何强化学习奖励 (Geometric Reward)²³。模型根据生成的 2D/3D 几何体与目标形状之间的 Chamfer Distance 获得连续衰减的强化学习奖励, 使大模型能够在多轮训练中自发学会复杂的 2D 草图约束与几何图元对齐²⁴。同时, **Text2CAD-Bench** 等评估基准的提出, 将测试集扩展至包含扫掠 (Sweep)、放样 (Loft)、抽壳 (Shell) 及复杂工程图纸标注等高级几何特征, 全面评估大模型的几何推理能力²⁶。

开源Agent工具链与闭环校验生态

在开源工程落地方面, 基于 LLM Agent 的设计自动化工具链发展迅速⁵。GitHub 开源项目 earthtojake/text-to-cad (CAD Skills) 构建了一套标准化的 Agent 技能库⁶。该项目包含了专门针对二维矢量图纸的 skills/dxf 工具, 能够根据自然语言或图像请求自动生成排版、垫片、法兰盘及板材切割图纸⁶。此外, 系统集成了 skills/sendcutsend 验证工具, 可在代码输出后自动检验 DXF 图纸中的闭合回路、最小线宽与孔洞间距, 确保生成的图纸直接符合钣金制造标准⁶。以 **HarnessCAD**、**CadSmith** 与 **IntentForge** 为代表的系统则倡导“验证器优先” (Verifier-First) 范式²¹。Agent 生成 Python CAD 代码后, 系统会在后台无头环境中静默执行, 并调用几何引擎测量生成图纸的真实物理属性 (如面积、闭合性、干涉情况), 若未通过断言则自动进入反馈迭代循环, 直到生成完美符合工程约束的矢量图纸²¹。

项目 / 数据集名称	核心输出模态	包含数据规模	几何特征与操作覆盖	开源状态 / 部署方式
Text2CAD	离散命令序列 Token ¹⁹	~170K 模型 / ~660K 标注 ¹⁹	基础 Sketch + Extrude ¹⁹	开源数据与模型权重 ²²
Text-to-CadQ	CadQuery	170,000 代码-	参数化草图、	开源微调数据

Query	Python 代码 ¹	文本对 ¹	体素布尔运算 ⁸	集与模型 ¹
Text2CAD-Benchmark	包含代码与 STEP 基准 ²⁶	600 高质量工程模型 ²⁶	包含 Fillet, Sweep, Loft, Shell 等 ²⁶	开源 Benchmark ³¹
CAD Skills (text-to-cad)	DXF / STEP / URDF ⁶	Agent Skills 工具包 ⁶	2D DXF 切割布局、3D 装配体 ⁶	MIT 协议, 支持 Skills CLI/Plugin ⁶
IntentForge / CadSmith	可验证 CadQuery 脚本 ²¹	自我纠错 Agent Harness ²¹	含 Pytest 断言校验的参数化图形 ²¹	开源 Agent 工具链 ²¹

面向工业级DXF与DWG生成的智能系统架构设计

基于对底层解析库、代码化建模引擎及AI大模型范式的深入分析，构建一个高可用、生产级的 Text-to-DXF/DWG 生成系统应当摒弃“端到端生成原始坐标”的脆弱模式，转而采用“自然语言解析 - 高层代码合成 - 沙盒执行与拓扑校验 - 视觉反馈与格式转换”的四层 Agent 闭环架构。

在系统的工作流程中，第一层为意图解析与结构化规范表达层。系统接收到用户的自然语言指令后，首先由通用大语言模型进行设计意图抽取，将模糊的语言转化为包含明确物理尺寸、公差要求、图层命名规范（如轮廓线层、中心线层、尺寸标注层）以及图纸单位（mm/inch）的中间表示（CAD-IR 或 JSON Schema）。这一层的作用是消除自然语言中的歧义，补全工业设计中隐式存在的工程常识。

第二层为高层 Python 代码合成层。模型根据解构后的结构化规范，生成基于 build123d 的 2D Sketch 脚本。利用 build123d 极具表现力的代数运算符与显式几何图元类，大模型只需关注高层几何拓扑逻辑与尺寸参数，而将繁重的相切计算、圆弧交点解算与边界闭合性保障交由底层的 OpenCASCADE 工业级求解内核处理。这种设计从根本上规避了大模型生成绝对数值坐标时不可避免的截断与漂移误差。

第三层为沙盒执行与拓扑自动校验层。系统在隔离的 Python 沙盒环境中静默执行合成的代码。如果代码存在语法错误或违反了几何内核的约束规则（如线段自相交或非法切线），沙盒捕获的错误 Traceback 诊断日志会自动呈递给大模型，触发代码自我修复机制。在代码成功执行并导出 DXF 后，系统调用 ezdxf 配合 Shapely 对提取的矢量多边形进行拓扑有效性检查，确保所有的外轮廓与内孔洞均形成绝对封闭的回路，不留任何悬空线段。

第四层为多模态视觉检验与多格式导出层。利用 ezdxf.addons.drawing 渲染管线，系统将生成的 DXF 矢量图纸实时绘制为高分辨率栅格图像，并送入视觉语言模型（VLM）中进行多模态视觉核验。VLM 会检查生成的图形外观、相对位置与标注样式是否与用户的原始视觉意图一致。一旦核验通过，系统通过 ezdxf.addons.odafc 调用后台无头运行的 ODA File Converter，将校验无误的 DXF 文件无损转换为工业界广泛使用的 DWG 二进制格式文件，完成整个自动化生成链路。

总结与工程落地指南

在开发面向 DXF/DWG 场景的 Text-to-CAD 项目时，技术选型与工程实施应重点把握以下核心方

向：

在底层解析与转换方面，推荐选用 MIT 协议授权的 ezdxf 作为基础的矢量数据操作与渲染库。若业务场景强依赖 DWG 格式，应避免存在 GPL 协议传染风险的纯开源解析库，采取通过 ezdxf.addons.odafc 绑定无头 ODA File Converter 的工业通用方案，实现 DXF 与 DWG 之间的高可靠无损转换。

在生成范式与建模引擎方面，应当坚决摒弃直接预测绝对坐标 Token 的路线，将 **build123d** 作为大模型代码合成的目标 DSL（领域专用语言）。build123d 严谨的 Python 上下文构建模式与代数运算符，既能够大幅降低大模型的代码生成门槛，又能借助 OpenCASCADE 内核确保生成的矢量图形具备绝对的拓扑闭合性与几何精确度。

在模型训练与 Agent workflow 构建方面，可借鉴 **Text-to-CadQuery** 与 **CAD-Coder** 的开源微调范式，使用 Qwen2.5-Coder 等优质开源代码基座模型，在“文本-代码”数据集上进行针对性微调。同时，系统必须搭建包含沙盒报错重试、Shapely 拓扑闭合断言以及 VLM 视觉核验的多重反馈闭环，从而确保输出的 DXF/DWG 图纸能够直接对接下游的激光切割、钣金加工与 CNC 数控生产系统。

Works cited

1. Paper page - Text-to-CadQuery: A New Paradigm for CAD Generation with Scalable Large Model Capabilities - Hugging Face, <https://huggingface.co/papers/2505.06507>
2. Text-to-CAD Generation Through Infusing Visual Feedback in Large Language Models, <https://icml.cc/virtual/2025/poster/46007>
3. Large Language Models for Computer-Aided Design: A Survey - arXiv, <https://arxiv.org/html/2505.08137v2>
4. DXF/DWG解析の時に使えるezdxfを使った便利技 #Python - Qiita, https://qiita.com/hk10_it/items/58c1c8106d46b900a797
5. From text to design: a framework to leverage LLM agents for automated CAD generation, https://www.researchgate.net/publication/395007339_From_text_to_design_a_framework_to_leverage_LLM_agents_for_automated_CAD_generation
6. GitHub - earthtojake/text-to-cad: A library of agent skills for CAD, CAE and CAM, <https://github.com/earthtojake/text-to-cad>
7. r/Text2CAD - Reddit, <https://www.reddit.com/r/Text2CAD/>
8. Text-to-CadQuery: A New Paradigm for CAD Generation with Scalable Large Model Capabilities - arXiv, <https://arxiv.org/html/2505.06507v1>
9. Text2CAD: Generating Sequential CAD Designs from Beginner-to-Expert Level Text Prompts | Request PDF - ResearchGate, https://www.researchgate.net/publication/397197580_Text2CAD_Generating_Sequential_CAD_Designs_from_Beginner-to-Expert_Level_Text_Prompts
10. mlightcad/awesome-cad: A curated list of valuable open-source CAD (Computer-Aided Design) software, libraries, and tools, organized by category. - GitHub, <https://github.com/mlightcad/awesome-cad>
11. ezdxfとopenCVで画像認識してCAD化する(ezdxfシリーズ番外編) - Qiita, <https://qiita.com/Rai-see/items/dc0fe559b733e715addb>

12. How to handle DWG files in C++ - Stack Overflow, <https://stackoverflow.com/questions/22597111/how-to-handle-dwg-files-in-c>
13. ODA Online File Converter | Open Design Alliance, https://www.opendesign.com/oda_online_converter
14. DWGファイルをDXFファイルへ変換するツールのインストールと使い方, https://user07.o-seven.info/manual/media/niwa_navi/20190620_1715_31_0552.pdf
15. GitHub - gumyr/build123d: A python CAD programming library | daily.dev, <https://daily.dev/posts/github---gumyr-build123d-a-python-cad-programming-library-s1iw3vdez>
16. TF01 Build123D - CAEなど工房, <https://enginfo.jp/tf01-build123d/>
17. build123d - PyPI, <https://pypi.org/project/build123d/>
18. build123d入門: PythonのコードでCADモデルを作って3Dプリントする - Zenn, https://zenn.dev/hiro_hamada/articles/build123d-python-cad-intro
19. NeurIPS Poster Text2CAD: Generating Sequential CAD Designs from Beginner-to-Expert Level Text Prompts, <https://neurips.cc/virtual/2024/poster/96571>
20. Text2CAD: Generating Sequential CAD Models from Beginner-to-Expert Level Text Prompts, <https://arxiv.org/html/2409.17106v1>
21. text-to-cad · GitHub Topics, <https://github.com/topics/text-to-cad?l=python&o=desc&s=updated>
22. Text2CAD — NeurIPS 2024 Spotlight - GitHub Pages, <https://sadiilkhan.github.io/text2cad-project/>
23. gudo7208/awesome-ai4cad: Survey & curated paper list: AI meets CAD — 700+ papers across generation, understanding, optimization, and manufacturing (2018–2026). - GitHub, <https://github.com/gudo7208/awesome-ai4cad>
24. CME-CAD: Heterogeneous Collaborative Multi-Expert Reinforcement Learning for CAD Code Generation - CVF Open Access, https://openaccess.thecvf.com/content/CVPR2026/papers/Niu_CME-CAD_Heterogeneous_Collaborative_Multi-Expert_Reinforcement_Learning_for_CAD_Code_Generation_CVPR_2026_paper.pdf
25. 1 Comparison of existing computer-use paradigms and our proposed ComAct paradigm. GUI-based agents rely on fragile visual grounding and suffer from long-horizon error accumulation, while API-based agents are constrained by fragmented and limited interfaces. In contrast, we leverage the COM as a unified semantic programmatic interface, enabling executable program synthesis - arXiv, <https://arxiv.org/html/2606.13239>
26. Text2CAD-Bench: A Benchmark for LLM-based Text-to-Parametric CAD Generation - arXiv, <https://arxiv.org/html/2605.18430v1>
27. YanjieZe/Paper-List: A paper list of my history reading. Robotics, Learning, Vision. - GitHub, <https://github.com/YanjieZe/Paper-List>
28. badele/all-stars - GitHub, <https://github.com/badele/all-stars>
29. [NeurIPS'24 Spotlight] Text2CAD: Generating Sequential CAD Designs from Beginner-to-Expert Level Text Prompts - GitHub, <https://github.com/SadiilKhan/Text2CAD>
30. Daily Papers - Hugging Face, <https://huggingface.co/papers?q=CAD%20text>

31. [2605.07807] Text-to-CAD Evaluation with CADTests - arXiv,
<https://arxiv.org/abs/2605.07807>